

PROJECT REPORT

Design and Implementation of an In-Vehicle Infotainment (IVI) System Integrating Media, Navigation and Projection

Conceptual architecture, interface annotation, data-flow design and a working Java simulator

Submitted by: Sanjoy Das

Enrollment no.: 12023002001304

Institution: Institute of Engineering and Management, Newtown

Course: B.Tech

Department: CSE

Roll: C-49

Date: 25 September 2026

Abstract

An In-Vehicle Infotainment (IVI) system brings entertainment, route guidance and smartphone content together on the vehicle head unit. The main engineering difficulty is not any single feature but the way features share scarce resources such as speakers and the display. This project designs a layered conceptual architecture for an IVI system that integrates media, navigation and smartphone projection (Android Auto and Apple CarPlay style), annotates the interfaces between components, and documents the data flows.

The architecture is validated with a working simulator written in Java (Swing, no external libraries). The simulator implements a media service, a navigation service with Dijkstra route calculation and turn-by-turn prompts, a projection session state machine, a publish/subscribe event bus that plays the role of the vehicle middleware, and an audio focus manager that arbitrates between features. Eleven automated tests pass, and screenshots of the running system demonstrate the key cross-feature behaviours: navigation prompts duck the media volume, and projected phone audio pauses the media player and restores it on disconnect.

Keywords: In-vehicle infotainment, IVI, system architecture, audio focus, Android Auto, Apple CarPlay, Dijkstra, Java Swing.

Table of contents

Abstract	2
1. Introduction	4
1.1 What an IVI system is and why it matters	4
1.2 Objective of the chosen option	4
1.3 Scope and limitations	4
2. Background and literature	6
2.1 IVI platforms and architectures	6
2.2 Smartphone projection	6
2.3 Audio management and audio focus	6
2.4 Route planning	6
2.5 Positioning of this work	6
3. Tools and environment	7
3.1 Tools used	7
3.2 Runtime environment	7
4. System design and methodology	8
4.1 Methodology	8
4.2 Functional requirements	8
4.3 Non-functional requirements	8
4.4 Conceptual architecture	8
4.5 Detailed design	10
5. Implementation and modelling	14
5.1 Modelling approach	14
5.2 Project structure	14
5.3 Key code excerpts	14
5.4 Building and running	15
6. Testing and results	16
6.1 Automated tests	16
6.2 Screenshots of the working model	16
6.3 Observed behaviour	19
7. Discussion	21
7.1 Interpretation of the results	21
7.2 Design decisions and trade-offs	21
7.3 Simulation compared with a real system	21
7.4 Observations	21
8. Conclusion and future scope	22
8.1 Conclusion	22
8.2 Future scope	22
References	23
Appendix A: Complete source code	24
Appendix B: Detailed reference architecture	41

1. Introduction

1.1 What an IVI system is and why it matters

An In-Vehicle Infotainment (IVI) system is the software and hardware platform behind the vehicle head unit. It combines media playback, navigation, phone integration and vehicle settings behind a touchscreen, steering-wheel controls and voice input. It is usually built on Linux, QNX or Android Automotive OS and must cooperate with vehicle networks, sensors, audio and display hardware, and the driver's smartphone.

The IVI system matters for three main reasons:

- **Driver experience:** the head unit is the part of the car the driver interacts with most. Responsiveness, a consistent interface and seamless hand-over between music, navigation and calls strongly shape how the whole vehicle is judged.
- **Safety:** the driver's attention must stay on the road. Voice prompts, glanceable screens and reliable audio arbitration (a turn instruction must always be audible over music) reduce distraction. Poorly coordinated features force the driver to look at the screen or miss guidance.
- **Connectivity:** the head unit connects to the smartphone (Android Auto, Apple Car-Play), the cloud (maps, traffic, streaming, software updates) and the vehicle network (speed, gear, sensors). The architecture must manage all of these links securely and consistently.

1.2 Objective of the chosen option

The chosen assignment option is: design a conceptual architecture diagram of an IVI system integrating media, navigation and projection features, annotate the component interfaces and data flow, and support the design with a working project, screenshots and code. The objectives are to:

- produce a layered conceptual architecture covering media, navigation and projection;
- annotate every interface between components and describe the data flows;
- implement the architecture as a Java simulator whose classes map onto the architectural components;
- demonstrate and test cross-feature behaviour, especially audio arbitration;
- document design, code, results and screenshots in this report.

1.3 Scope and limitations

In scope: the HMI layer, the three feature services (media, navigation, projection), and the shared platform services (event bus, audio focus manager, routing engine), all running as a Java desktop simulation.

Limitations: this is a simulation only.

- There is no real vehicle bus: CAN, automotive Ethernet and SOME/IP are represented by an in-process event bus.
- Android Auto and Apple CarPlay are not used. They require certified hardware and licences, so projection is a session state machine with a mock phone screen.
- Media playback is timer-based and produces no real audio; navigation uses a small road graph with simulated position instead of GNSS and real map data.

- Security, over-the-air updates, functional safety (ISO 26262) and timing performance are outside the scope.

2. Background and literature

2.1 IVI platforms and architectures

Production IVI systems follow a few dominant approaches. Android Automotive OS runs a full Android-based operating system on the head unit, with a Vehicle HAL that exposes vehicle data to applications [2]. Linux-based platforms, promoted by the COVESA (formerly GENIVI) community, use open middleware and a layered stack [6]. Service-oriented designs based on the AUTOSAR Adaptive Platform use SOME/IP to let software components discover and call each other across ECUs [7]. Despite the differences, they share the same idea: applications sit on top of platform services, which sit on hardware abstraction layers. The architecture in this report follows that layering.

2.2 Smartphone projection

Android Auto [4] and Apple CarPlay [5] let the phone run the application and render its interface on the vehicle display, while touch input is sent back to the phone. Connections use USB or a wireless link. The head unit still owns the vehicle-side responsibilities: the display, audio routing, input devices and integration with native features. Projection therefore has to be managed as a session with a lifecycle, and its audio must be coordinated with native media and navigation. This is the role of the `ProjectionManager` in this project.

2.3 Audio management and audio focus

When several sources want to use the speakers, a policy is needed. Android's audio focus model [3] is a widely used example: an application requests focus, a transient interruption can cause other sources to lower their volume (ducking), and a longer interruption causes them to pause until focus returns. This project adopts the same three outcomes, called GAIN, DUCK and LOSS, in its `AudioFocusManager`.

2.4 Route planning

Navigation systems represent the road network as a weighted graph and compute routes with shortest-path algorithms. Dijkstra's algorithm [1] finds the minimum-cost path from a source to all reachable nodes and is the basis of many routing methods. Production systems extend it with heuristics such as A* and precomputed hierarchies for speed on continent-scale maps. For the small graph used here, plain Dijkstra with a priority queue is sufficient.

2.5 Positioning of this work

Full IVI platforms are large and depend on proprietary components, so they are difficult to study in a course project. This work isolates what matters architecturally: layered structure, well-defined interfaces, an event-based middleware and cross-feature arbitration. It presents them in a small, testable simulator that can run on any computer with a JDK.

3. Tools and environment

3.1 Tools used

Tool	Version / detail	Purpose
Java Development Kit	JDK 11 or newer (tested with JDK 21)	Compile and run the simulator and tests
Java Swing and Java2D	Part of the JDK	User interface, map and phone screen drawing
Visual Studio Code	Extension Pack for Java	Editing, running, integrated terminal
Shell scripts	run.sh (macOS/Linux), run.bat (Windows)	Build, run, test and screenshot commands
macOS	MacBook Air	Development and test machine
Python with Matplotlib	3.x	Architecture, class, sequence and state diagrams
Eraser	Diagram tool	Detailed reference architecture (Appendix B)
Microsoft Word	-	Report

3.2 Runtime environment

The simulator is a single-process desktop application. A Swing timer advances the simulation every 250 ms on the UI thread, so services and screens run in a deterministic order without threading issues. The core package has no UI dependency, so the same code runs in the automated tests and in a headless mode (`java.awt.headless=true`) that renders the screenshots used in this report. No vehicle hardware, phone, network or third-party library is required.

4. System design and methodology

4.1 Methodology

The design follows a top-down method. Requirements were written first (4.2 and 4.3). A layered conceptual architecture with annotated interfaces and data flows was then derived (4.4). Each component was refined into a class design with defined behaviour (4.5), implemented as a Java simulator (Section 5), and finally verified with automated tests and screenshots (Section 6).

4.2 Functional requirements

ID	Requirement
FR1	The system shall play, pause, skip and select tracks from a playlist and allow the volume to be changed.
FR2	The system shall compute the shortest route between two places on a road network and display it on a map.
FR3	The system shall simulate vehicle position along the route and issue turn-by-turn voice prompts before each junction.
FR4	The system shall support a smartphone projection session with the states disconnected, handshake and streaming.
FR5	While projection is streaming, the user shall be able to select phone applications from the projected screen.
FR6	The system shall arbitrate audio: navigation prompts lower the media volume, and projected phone music pauses media.
FR7	Media shall resume automatically when the interrupting audio source ends.
FR8	All inter-component events shall be visible in an event log to illustrate the data flows.

4.3 Non-functional requirements

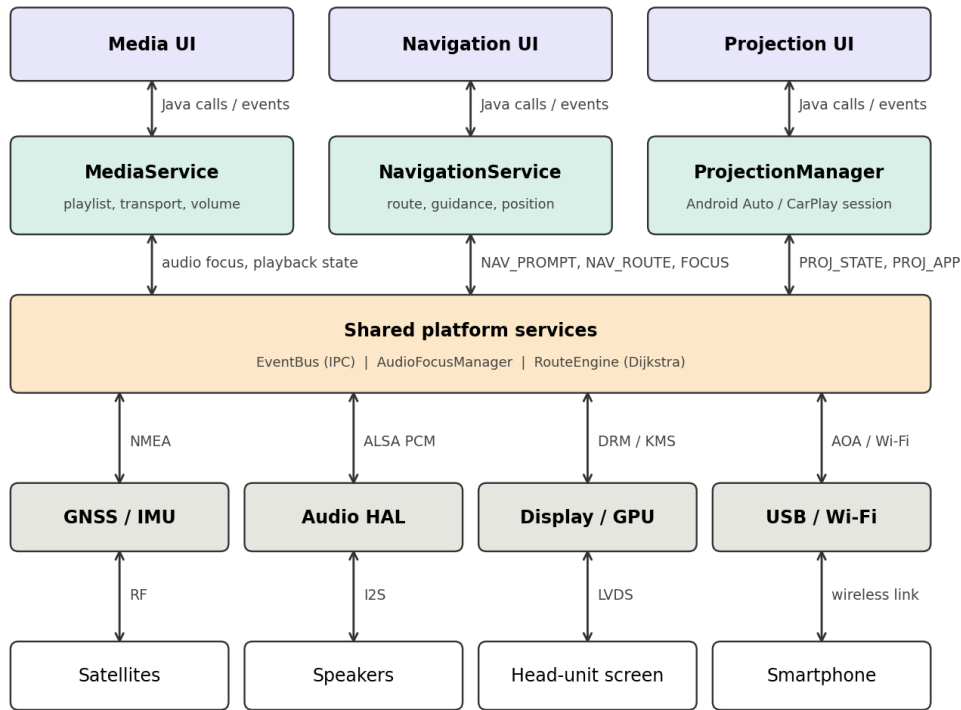
- **Modularity:** features are separate services that communicate through defined interfaces and an event bus.
- **Portability:** pure Java with the Swing toolkit, running on Windows, macOS and Linux without extra libraries.
- **Testability:** core logic is free of UI code so it can be tested headlessly; screenshots can be rendered without a display.
- **Responsiveness:** the simulation runs on a 250 ms clock and updates all screens on every step.

4.4 Conceptual architecture

Overview

The system is organised in five layers (Figure 1). The HMI layer contains the three user-facing applications. Below it, one service per feature implements the business logic. A shared platform layer provides the services that all features depend on: an event bus that stands in for the vehicle middleware, an audio focus manager and a routing engine. Drivers and external devices sit at the bottom. Every arrow is annotated with the interface or protocol carried on it.

Conceptual IVI architecture: media, navigation and projection



Implemented in the Java simulator: HMI layer, services and shared services. Drivers and external items are simulated.

Figure 1: Layered conceptual architecture with annotated interfaces

Layer responsibilities

Layer	Responsibility
HMI layer	Media UI, Navigation UI and Projection UI. Presents state and forwards touch input to the services.
Services	MediaService (playlist, transport, volume), NavigationService (route following, position, prompts), ProjectionManager (phone session lifecycle).
Shared platform services	EventBus (inter-component messaging), AudioFocusManager (audio arbitration), RouteEngine (shortest-path routing).
Drivers / HAL	GNSS/IMU, audio HAL, display/GPU and USB/Wi-Fi stacks. Simulated in this project.
External	Satellites, speakers, head-unit screen and the smartphone.

Component interfaces

Table 1 lists each interface, the components on either side, the data exchanged and the real-world technology it represents.

ID	Interface	Between	Data / real-world equivalent
I1	Media control API	MediaPanel to Media-Service	play, pause, next, prev, select, setVolume. Equivalent to Android MediaSession / AIDL.
I2	Navigation API	NavPanel to NavigationService	startRoute(dest), cancel(). Equivalent to the Android Car App Library navigation interfaces.
I3	Projection API	ProjectionPanel to ProjectionManager	connect(protocol), launchApp(name), disconnect(). Equivalent to an Android Auto / CarPlay session.

ID	Interface	Between	Data / real-world equivalent
I4	Audio focus API	Services to AudioFocusManager	request(Source), release(Source). Equivalent to Android AudioManager audio focus.
I5	Event bus	All services	publish(topic, payload), subscribe(topic, handler). Equivalent to SOME/IP, D-Bus or binder IPC.
I6	Routing API	NavigationService to RouteEngine	shortestPath(from, to) returning ordered node names. Equivalent to a map/routing service.
I7	Position source	NavigationService (simulated)	x, y position along the route. Equivalent to NMEA data from a GNSS receiver.

Table 1: Component interfaces

Data flows

ID	Flow	Source to destination	Bus topic / data
F1	User commands	UI panels to services	Direct method calls (I1, I2, I3)
F2	Route announced	NavigationService to EventBus	nav.route: list of nodes
F3	Voice prompt	NavigationService to EventBus	nav.prompt: prompt text
F4	Focus request / release	NavigationService, ProjectionManager to AudioFocusManager	request / release(Source)
F5	Focus change	AudioFocusManager to MediaService via EventBus	audio.focus: GAIN, DUCK or LOSS
F6	Media state	MediaService to EventBus	media.state: playing, paused, track change
F7	Projection state	ProjectionManager to EventBus	projection.state, projection.app
F8	Screen refresh	Services to UI panels	Panels read service state every 250 ms step

Table 2: Data flows

A more detailed reference architecture that also shows the vehicle network, cloud services and security boundary is included as Appendix B. The simulator implements the subset that is relevant to the three required features.

4.5 Detailed design

Class structure

Each architectural component is one Java class (Figure 2). The core package has no dependency on Swing, which keeps the logic testable. The ui package contains one panel per application plus IviSystem, which wires the services together and advances the simulation.

Class structure and dependencies

Arrows show 'uses' relationships. MediaService, NavigationService and ProjectionManager also publish events to EventBus.

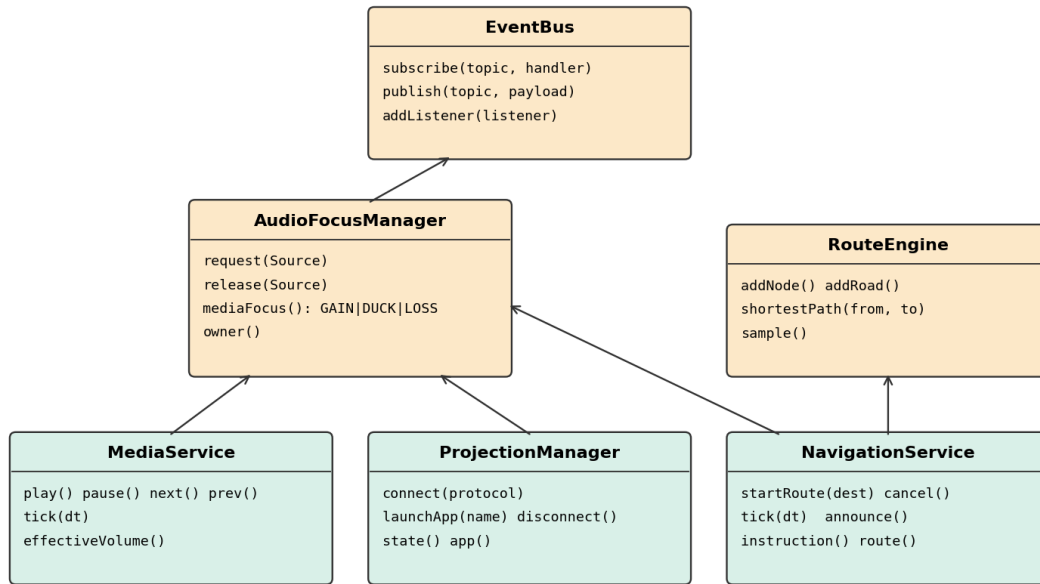


Figure 2: Class structure and dependencies

Event bus

EventBus keeps a map from topic name to a list of handlers. A service calls `publish(topic, payload)` and every subscribed handler is invoked synchronously. A separate listener interface receives all events and feeds the on-screen log. Topic names are defined once in the Topics class, so each annotated data flow in the architecture corresponds to exactly one constant.

Audio focus arbitration

Audio is the shared resource that couples the three features. AudioFocusManager holds the set of sources that currently want to play and derives the state of the media player from it using fixed priorities:

Active sources	Media focus	Effect on media
None (or only MEDIA)	GAIN	Plays at the user volume.
NAV_PROMPT	DUCK	Volume reduced to 30 percent while the prompt is spoken.
PROJECTION	LOSS	Playback paused; resumes automatically on GAIN.
PROJECTION and NAV_PROMPT	LOSS	Projection has the highest priority.

Table 3: Audio focus rules

The focus manager publishes a FOCUS event only when the derived state changes. MediaService subscribes to it and remembers whether it was paused by focus loss, so a manual pause is never overridden by an automatic resume.

Sequence: navigation prompt ducks media

Figure 3 shows the interaction that occurs when the vehicle approaches a junction while music is playing.

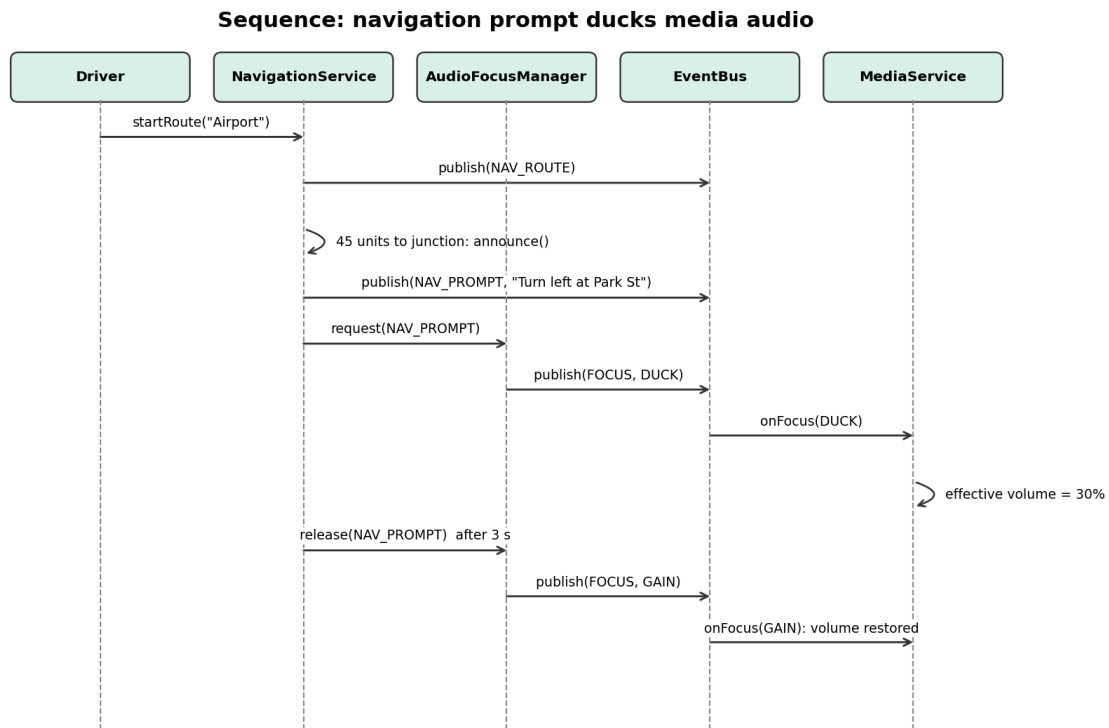


Figure 3: Navigation prompt ducking media audio

Route calculation and guidance

The road network is a weighted undirected graph whose nodes carry map coordinates and whose edge weights are Euclidean distances. `RouteEngine.shortestPath` implements Dijkstra's algorithm with a priority queue, giving a running time of $O((V + E) \log V)$. For the sample network the route from Home to Airport is Home, Park St, School, Mall, Stadium, Airport.

`NavigationService` moves the simulated vehicle along the route at constant speed. When the remaining distance to the next junction falls below 45 map units it generates a prompt. The turn direction is derived from the sign of the two-dimensional cross product of the incoming and outgoing road vectors (screen coordinates, y pointing down): a positive value means turn right, a negative value means turn left, and a value near zero means continue straight. The last segment produces a "destination is ahead" prompt.

Projection session state machine

`ProjectionManager` models the lifecycle of a phone session (Figure 4). A connect request starts a two second handshake, after which the session streams. While streaming, selecting the Music tile requests PROJECTION audio focus, and selecting any other tile releases it. Disconnecting releases the focus and returns to the disconnected state.

ProjectionManager state machine

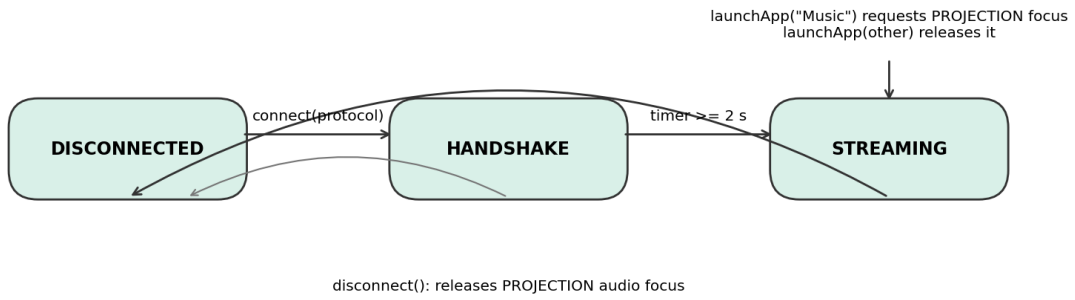


Figure 4: ProjectionManager state machine

User interface

The head-unit window has three tabs (Media, Navigation, Projection), a status line showing the current audio owner and media focus, and an event bus log at the bottom. The status line and log make the otherwise invisible architecture behaviour visible in every screenshot.

5. Implementation and modelling

5.1 Modelling approach

Each real-world element is modelled by one class, keeping the behaviour that matters for the architecture and simplifying the rest (Table 5). The tools used are listed in Section 3.

Real element	Modelled by	Simplification
Vehicle middleware (SOME/IP, D-Bus)	EventBus	In-process, synchronous delivery
Audio manager / audio HAL	AudioFocusManager	Focus state only; no sound output
Media framework	MediaService	Timer-based playback of a four-track playlist
GNSS, map and routing service	NavigationService, RouteEngine	Constant speed along a small road graph
Phone session (Android Auto / CarPlay)	ProjectionManager	State machine with a mock phone screen
Head-unit display and input	Swing panels	Mouse replaces touch

Table 5: How real components are modelled

5.2 Project structure

```
ivi/
  run.sh, run.bat, README.md
  src/ivi/core/    EventBus, Topics, AudioFocusManager, MediaService,
                  RouteEngine, NavigationService, ProjectionManager
  src/ivi/ui/      MediaPanel, NavPanel, ProjectionPanel, IviSystem, MainApp
  src/ivi/test/    IviTests, ScreenshotTool
  screenshots/    generated PNG files
```

5.3 Key code excerpts

The following excerpts show the three central pieces of logic. The complete source is listed in Appendix A.

Audio focus arbitration

```
private void update() {
    Focus f = active.contains(Source.PROJECTION) ? Focus.LOSS
        : active.contains(Source.NAV_PROMPT) ? Focus.DUCK : Focus.GAIN;
    if (f != mediaFocus) {
        mediaFocus = f;
        bus.publish(Topics.FOCUS, f);
    }
}
```

Reaction of the media service to focus changes

```
private void onFocus(AudioFocusManager.Focus f) {
    if (f == AudioFocusManager.Focus.LOSS && playing) { playing = false; pausedByFocus = true; }
    else if (f == AudioFocusManager.Focus.GAIN && pausedByFocus) { playing = true; pausedByFocus = false; }
    publishState("focus " + f);
}
```

Turn direction from the cross product

```
double cross = (b.x - a.x) * (c.y - b.y) - (b.y - a.y) * (c.x - b.x);
String turn = Math.abs(cross) < 1 ? "Continue straight"
    : cross > 0 ? "Turn right" : "Turn left";
```

5.4 Building and running

On macOS with a JDK installed, open the project folder in VS Code and run the following in the integrated terminal:

```
chmod +x run.sh
./run.sh          # start the simulator
./run.sh test     # run the automated tests
./run.sh shots    # regenerate the screenshots
```

6. Testing and results

6.1 Automated tests

IviTests contains eleven assertion-based tests of the core logic. They run without a display and were executed on macOS with JDK 21; all passed.

#	Test	Result
1	Dijkstra returns Home, Park St, School, Mall, Stadium, Airport	PASS
2	Media plays at full volume when no other source is active	PASS
3	A navigation prompt ducks media (volume 60 becomes 18)	PASS
4	Media volume is restored after the prompt is released	PASS
5	Projection moves to HANDSHAKE after connect	PASS
6	Projection moves to STREAMING after two seconds	PASS
7	Phone music pauses IVI media	PASS
8	Media resumes after projection disconnect	PASS
9	Navigation reaches the destination and reports arrival	PASS
10	At least four turn-by-turn prompts are issued (five ob-	PASS
11	Audio focus returns to GAIN after all prompts end	PASS

Table 4: Automated test cases

Console output of the test run:

```
PASS Dijkstra Home->Airport
PASS media plays at full volume
PASS nav prompt ducks media
PASS media volume restored
PASS projection handshake
PASS projection streaming
PASS phone music pauses IVI media
PASS media resumes after disconnect
PASS navigation reaches destination
PASS turn-by-turn prompts issued (5)
PASS focus released after prompts
11 tests passed
```

6.2 Screenshots of the working model

The screenshots below show the working simulator. Each one demonstrates a specific part of the architecture. Figures 5 to 10 were produced by the ScreenshotTool class, which renders the real user interface to image files.

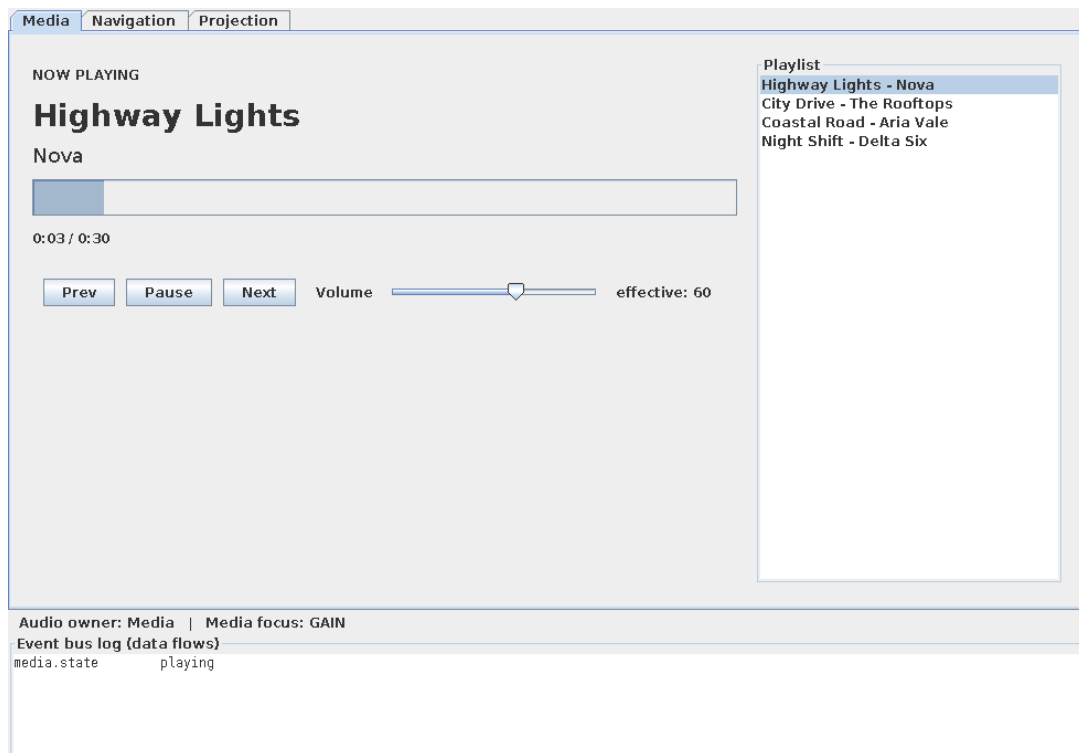


Figure 5: Media application playing a track (playlist, progress bar, volume).

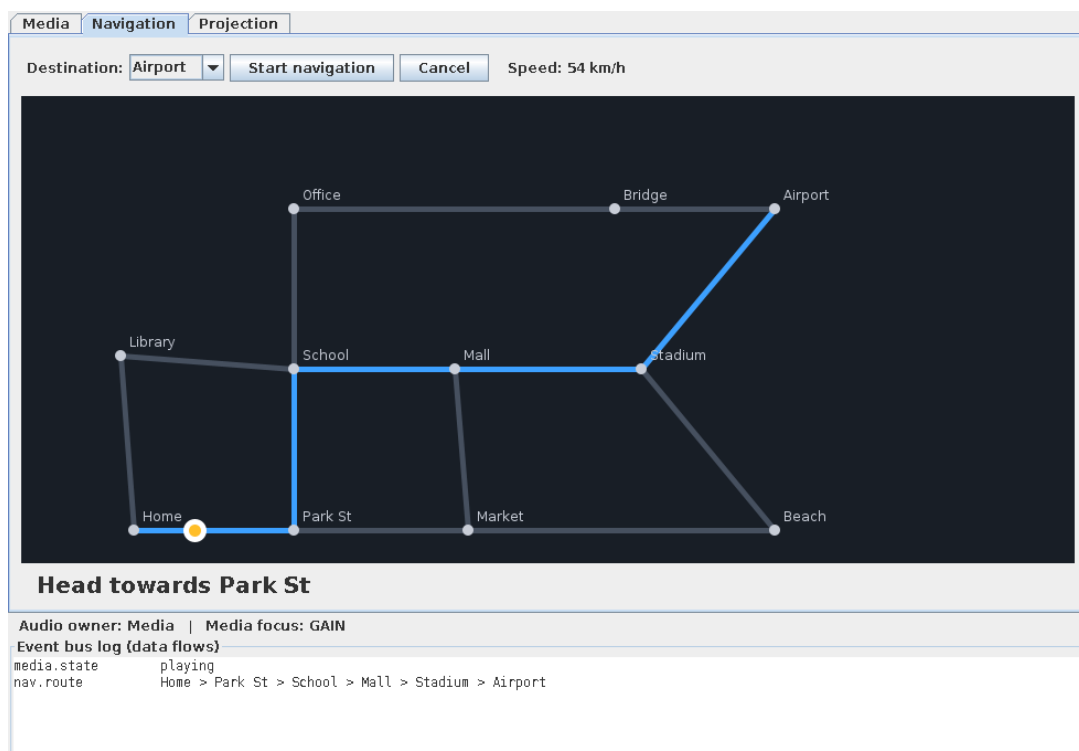


Figure 6: Navigation application after starting a route from Home to Airport; route highlighted in blue.

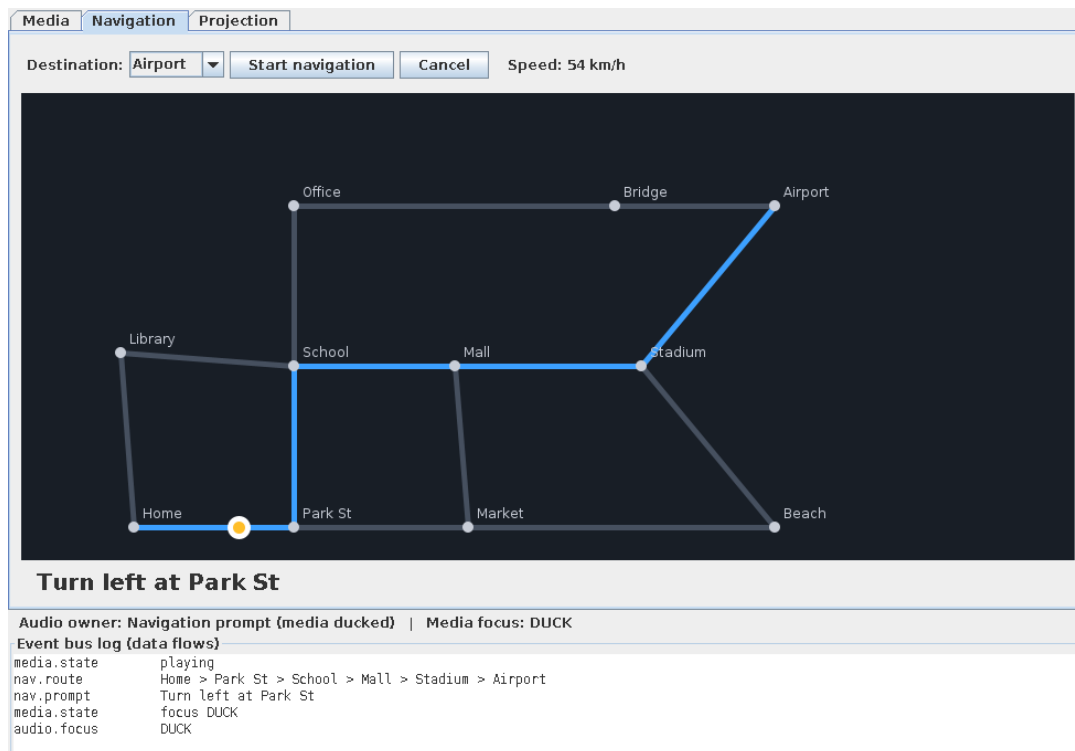


Figure 7: Turn prompt "Turn left at Park St" issued; status bar shows media ducked and the event log shows the FOCUS event.

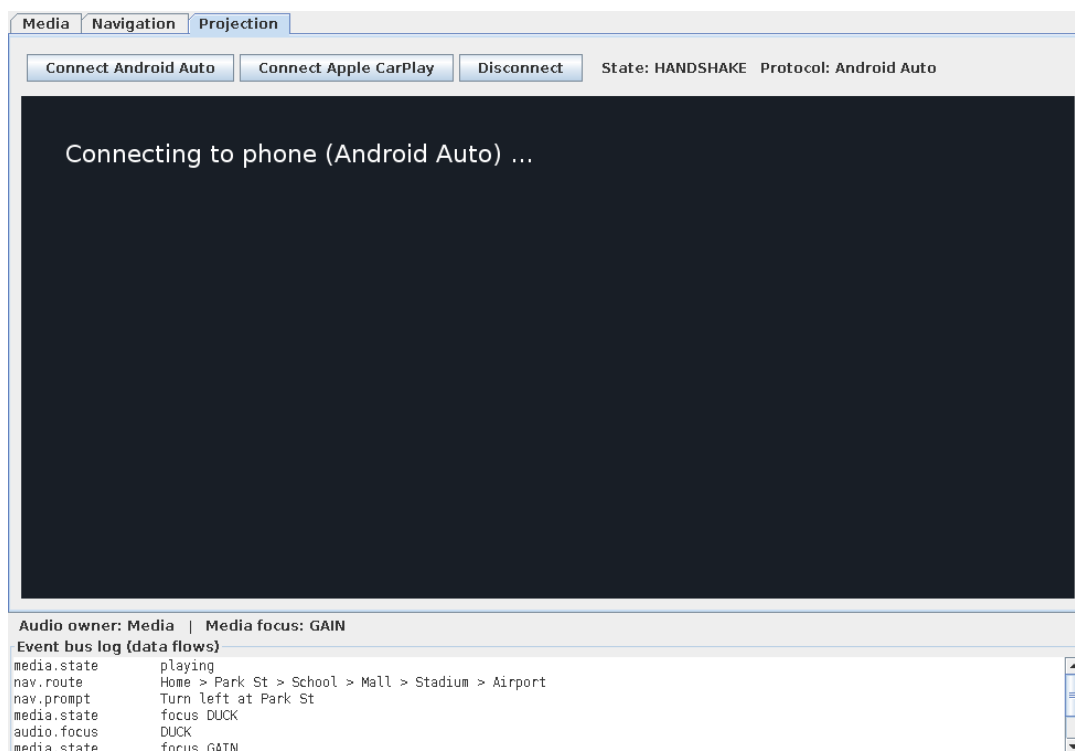


Figure 8: Projection screen during the HANDSHAKE state (Android Auto).

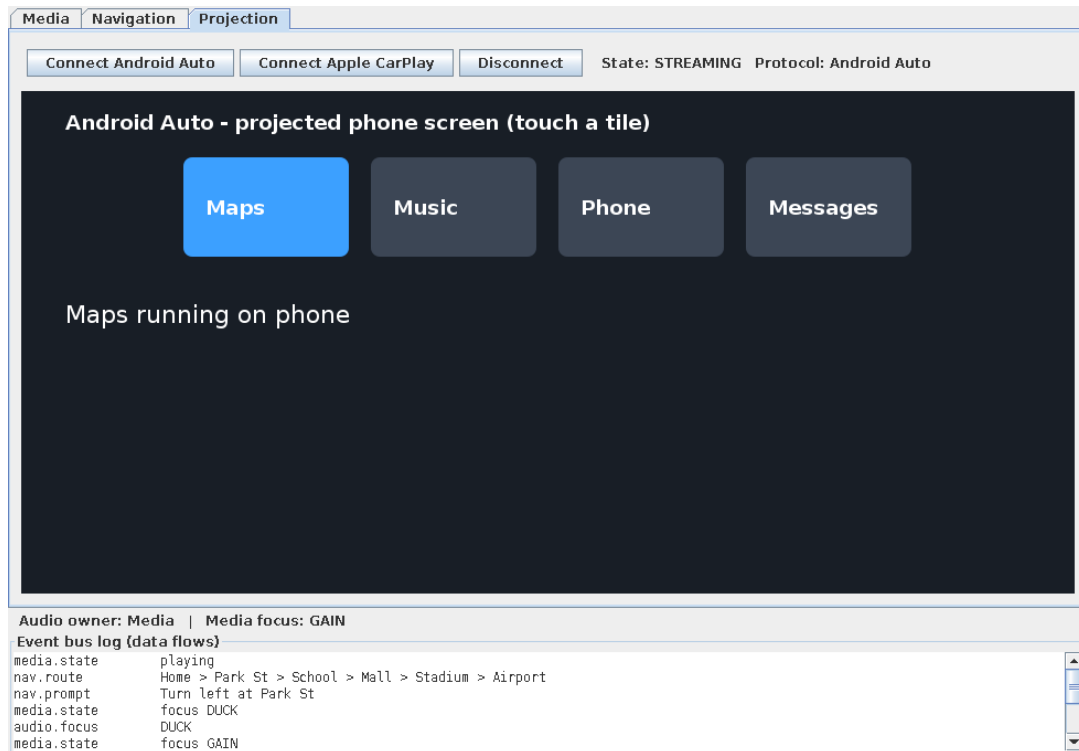


Figure 9: Projection screen in the STREAMING state with the Maps app selected.

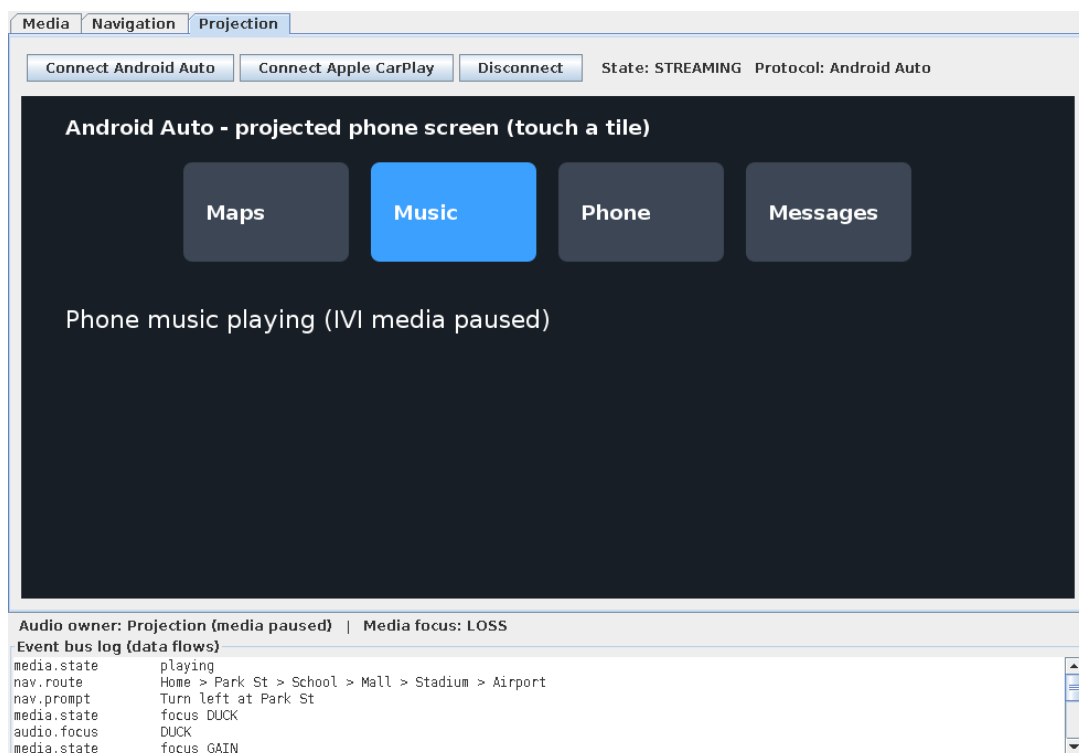


Figure 10: Phone Music selected: projection owns audio and the IVI media is paused.

6.3 Observed behaviour

- Starting navigation publishes the route on the nav.route topic and the map highlights the road from Home to Airport.
- At each junction the navigation service publishes a prompt and requests audio focus. The event log then shows audio.focus DUCK, and the status bar reads "Navigation prompt (media ducked)".

- About three seconds later the prompt ends, the focus is released and the media volume returns to its previous level.
- Connecting a phone moves the session from HANDSHAKE to STREAMING after two seconds. Selecting Music on the phone pauses the IVI media player; disconnecting resumes it.

7. Discussion

7.1 Interpretation of the results

The results show that the central architectural idea works: features stay independent, yet they behave correctly together because they coordinate through one shared audio focus manager and one event bus. Navigation never needs to know that media exists; it only requests and releases NAV_PROMPT focus. Media never needs to know about navigation or projection; it only reacts to GAIN, DUCK and LOSS. This loose coupling is what allowed the cross-feature scenarios to be tested with a few lines of code (tests 3, 4, 7 and 8).

The screenshots confirm the same behaviour visually. The status line and event log make the invisible flows visible, showing FOCUS events at the moment a prompt starts and ends, and showing media paused while projected phone music plays.

7.2 Design decisions and trade-offs

- **In-process event bus:** chosen for simplicity and deterministic behaviour. It hides latency, message loss and process failure, which a real SOME/IP or binder deployment must handle.
- **Derived audio focus:** the focus state is computed from the set of active sources with fixed priorities. This is easy to reason about and test, but it has no per-application priorities and no queue of transient requests.
- **Polling UI on a 250 ms tick:** panels read service state every step rather than reacting to each event. This keeps the UI code simple at the cost of up to one tick of delay.
- **Constant-speed navigation:** a fixed speed makes runs reproducible for testing. Real guidance would use GNSS, IMU and vehicle speed signals.
- **Only the Music tile takes audio:** this keeps the projection demo clear. A real phone would also send navigation voice and call audio.

7.3 Simulation compared with a real system

The simulator reproduces the structure and the logic of an IVI system, not its physical or timing behaviour. Video decoding, audio latency, jitter, bandwidth limits of USB or Wi-Fi, vehicle bus signals, security enforcement and functional safety are not modelled. The results should therefore be read as validating the architecture and the arbitration rules, not the performance of a production system.

7.4 Observations

The main lesson is that the complexity of an IVI system comes less from any single feature than from interactions between features over shared resources. Making those interactions explicit, through named interfaces, named topics and one arbitration point, made the design easier to explain, implement and test.

8. Conclusion and future scope

8.1 Conclusion

This project produced a layered conceptual architecture for an IVI system integrating media, navigation and projection, with annotated interfaces and documented data flows. The architecture was validated by a Java simulator whose classes map directly onto the architectural components. A shared audio focus manager, coordinated through an event bus, lets independent features coexist: navigation prompts temporarily duck the media volume and projected phone audio pauses it, with automatic recovery afterwards. Eleven automated tests pass and the screenshots confirm the behaviour on screen.

8.2 Future scope

- Integrate real projection protocols and certified hardware, or port the design to Android Automotive OS using the Car App Library with the same service boundaries.
- Replace timer-based playback with a real media framework such as GStreamer or Android MediaCodec, with actual audio output.
- Use real map data and a routing service with traffic, and drive guidance from GNSS, IMU and vehicle speed signals.
- Replace the in-process event bus with SOME/IP, D-Bus or binder between separate processes, and add fault handling.
- Add the vehicle-network, security (secure boot, firewall, authentication) and over-the-air update parts shown in the reference architecture in Appendix B.
- Extend audio arbitration with per-source priorities and call audio, and add driver-distraction rules.

References

- [1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269-271.
- [2] Android Developers. Android Automotive OS and Car App Library documentation. developer.android.com/training/cars
- [3] Android Developers. Manage audio focus. developer.android.com/media/optimize/audio-focus
- [4] Google. Android Auto. android.com/auto
- [5] Apple. CarPlay. apple.com/ios/carplay
- [6] COVESA (formerly GENIVI). Open automotive software initiatives. covesa.global
- [7] AUTOSAR. Adaptive Platform and SOME/IP service-oriented communication. autosar.org
- [8] Oracle. Creating a GUI with Swing (Java Tutorials). docs.oracle.com/javase/tutorial/uiswing

Appendix A: Complete source code

All files are under src/ivi/. Files are listed in the order core, ui, test.

A.1 core/EventBus.java

Publish/subscribe bus simulating the middleware (IPC / D-Bus / SOME-IP).

```
package ivi.core;

import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.BiConsumer;
import java.util.function.Consumer;

/** In-process publish/subscribe bus. Simulates the IPC / D-Bus / SOME-IP middleware. */
public class EventBus {
    private final Map<String, List<Consumer<Object>>> subs = new HashMap<>();
    private final List<BiConsumer<String, Object>> listeners = new CopyOnWriteArrayList<>();

    public void subscribe(String topic, Consumer<Object> handler) {
        subs.computeIfAbsent(topic, k -> new CopyOnWriteArrayList<>()).add(handler);
    }

    /** Receives every event (used by the on-screen event log). */
    public void addListener(BiConsumer<String, Object> l) { listeners.add(l); }

    public void publish(String topic, Object payload) {
        for (Consumer<Object> h : subs.getOrDefault(topic, List.of())) h.accept(payload);
        for (BiConsumer<String, Object> l : listeners) l.accept(topic, payload);
    }
}
```

A.2 core/Topics.java

Topic names, one per annotated data flow.

```
package ivi.core;

/** Bus topic names = the annotated data flows in the architecture diagram. */
public final class Topics {
    private Topics() {}
    public static final String MEDIA_STATE = "media.state";
    public static final String FOCUS      = "audio.focus";
    public static final String NAV_ROUTE  = "nav.route";
    public static final String NAV_PROMPT = "nav.prompt";
    public static final String PROJ_STATE = "projection.state";
    public static final String PROJ_APP   = "projection.app";
}
```

A.3 core/AudioFocusManager.java

Shared service arbitrating audio between media, navigation prompts and projection.

```
package ivi.core;

import java.util.EnumSet;
import java.util.Set;

/**
 * Shared platform service that arbitrates audio between features.
 * Rules: PROJECTION audio pauses media (LOSS); NAV_PROMPT lowers media volume (DUCK).
```

```

*/
public class AudioFocusManager {
    public enum Source { MEDIA, NAV_PROMPT, PROJECTION }
    public enum Focus { GAIN, DUCK, LOSS }

    private final EventBus bus;
    private final Set<Source> active = EnumSet.noneOf(Source.class);
    private Focus mediaFocus = Focus.GAIN;

    public AudioFocusManager(EventBus bus) { this.bus = bus; }

    public void request(Source s) { active.add(s); update(); }
    public void release(Source s) { active.remove(s); update(); }
    public Focus mediaFocus() { return mediaFocus; }

    public String owner() {
        if (active.contains(Source.PROJECTION)) return "Projection (media paused)";
        if (active.contains(Source.NAV_PROMPT)) return "Navigation prompt (media ducked)";
        return "Media";
    }

    private void update() {
        Focus f = active.contains(Source.PROJECTION) ? Focus.LOSS
            : active.contains(Source.NAV_PROMPT) ? Focus.DUCK : Focus.GAIN;
        if (f != mediaFocus) {
            mediaFocus = f;
            bus.publish(Topics.FOCUS, f);
        }
    }
}

```

A.4 core/MediaService.java

Playlist, transport controls, volume and focus handling.

```

package ivi.core;

import java.util.List;

/** Media service: playlist, transport controls, volume; reacts to audio focus changes. */
public class MediaService {
    public static class Track {
        public final String title, artist;
        public final int seconds;
        public Track(String title, String artist, int seconds) {
            this.title = title; this.artist = artist; this.seconds = seconds;
        }
        @Override public String toString() { return title + " - " + artist; }
    }

    private final EventBus bus;
    private final AudioFocusManager focusMgr;
    private final List<Track> playlist = List.of(
        new Track("Highway Lights", "Nova", 30),
        new Track("City Drive", "The Rooftops", 25),
        new Track("Coastal Road", "Aria Vale", 35),
        new Track("Night Shift", "Delta Six", 28));
    private int index = 0;
    private double position = 0;
    private boolean playing = false, pausedByFocus = false;
    private int volume = 60;

```

```

public MediaService(EventBus bus, AudioFocusManager focusMgr) {
    this.bus = bus; this.focusMgr = focusMgr;
    bus.subscribe(Topics.FOCUS, p -> onFocus((AudioFocusManager.Focus) p));
}

public void play() {
    if (focusMgr.mediaFocus() == AudioFocusManager.Focus.LOSS) { publishState("blocked:
projection audio active"); return; }
    playing = true; publishState("playing");
}
public void pause() { playing = false; pausedByFocus = false; publishState("paused"); }
public void toggle() { if (playing) pause(); else play(); }
public void next() { select((index + 1) % playlist.size()); }
public void prev() { select((index - 1 + playlist.size()) % playlist.size()); }
public void select(int i) { index = i; position = 0; publishState("track: " + current());
}

public void setVolume(int v) { volume = Math.max(0, Math.min(100, v)); }

/** Advance playback time (called by the system clock). */
public void tick(double dt) {
    if (!playing) return;
    position += dt;
    if (position >= current().seconds) next();
}

private void onFocus(AudioFocusManager.Focus f) {
    if (f == AudioFocusManager.Focus.LOSS && playing) { playing = false; pausedByFocus =
true; }
    else if (f == AudioFocusManager.Focus.GAIN && pausedByFocus) { playing = true;
pausedByFocus = false; }
    publishState("focus " + f);
}

private void publishState(String why) { bus.publish(Topics.MEDIA_STATE, why); }

public Track current() { return playlist.get(index); }
public List<Track> playlist() { return playlist; }
public int index() { return index; }
public double position() { return position; }
public boolean isPlaying() { return playing; }
public boolean isPausedByFocus() { return pausedByFocus; }
public int volume() { return volume; }
public int effectiveVolume() {
    switch (focusMgr.mediaFocus()) {
        case DUCK: return (int) Math.round(volume * 0.3);
        case LOSS: return 0;
        default: return volume;
    }
}
}

```

A.5 core/RouteEngine.java

Road graph and Dijkstra shortest-path routing.

```
package ivi.core;

import java.util.*;

/** Road graph + Dijkstra shortest-path routing. */
public class RouteEngine {
    public static class Node {
        public final String name; public final double x, y;
        Node(String name, double x, double y) { this.name = name; this.x = x; this.y = y; }
    }

    private final Map<String, Node> nodes = new LinkedHashMap<>();
    private final Map<String, Map<String, Double>> adj = new HashMap<>();
    private final List<String[]> roads = new ArrayList<>();

    public void addNode(String name, double x, double y) {
        nodes.put(name, new Node(name, x, y));
        adj.put(name, new HashMap<>());
    }

    public void addRoad(String a, String b) {
        Node n1 = nodes.get(a), n2 = nodes.get(b);
        double d = Math.hypot(n1.x - n2.x, n1.y - n2.y);
        adj.get(a).put(b, d);
        adj.get(b).put(a, d);
        roads.add(new String[]{a, b});
    }

    /** Returns the ordered list of node names from 'from' to 'to', or empty if unreachable.
    */
    public List<String> shortestPath(String from, String to) {
        Map<String, Double> dist = new HashMap<>();
        Map<String, String> prev = new HashMap<>();
        PriorityQueue<Map.Entry<String, Double>> pq =
            new PriorityQueue<>(Map.Entry.comparingByValue());
        for (String n : nodes.keySet()) dist.put(n, Double.MAX_VALUE);
        dist.put(from, 0.0);
        pq.add(new AbstractMap.SimpleEntry<>(from, 0.0));
        while (!pq.isEmpty()) {
            Map.Entry<String, Double> cur = pq.poll();
            String u = cur.getKey();
            if (cur.getValue() > dist.get(u)) continue;
            if (u.equals(to)) break;
            for (Map.Entry<String, Double> e : adj.get(u).entrySet()) {
                double nd = dist.get(u) + e.getValue();
                if (nd < dist.get(e.getKey())) {
                    dist.put(e.getKey(), nd);
                    prev.put(e.getKey(), u);
                    pq.add(new AbstractMap.SimpleEntry<>(e.getKey(), nd));
                }
            }
        }
        if (dist.get(to) == Double.MAX_VALUE) return List.of();
        LinkedList<String> path = new LinkedList<>();
        for (String at = to; at != null; at = prev.get(at)) path.addFirst(at);
        return path;
    }
}
```

```

public Node node(String name) { return nodes.get(name); }
public Collection<Node> nodes() { return nodes.values(); }
public List<String[]> roads() { return roads; }

public static RouteEngine sample() {
    RouteEngine r = new RouteEngine();
    r.addNode("Home", 60, 300);    r.addNode("Park St", 180, 300);
    r.addNode("Market", 310, 300); r.addNode("Beach", 540, 300);
    r.addNode("Library", 50, 170); r.addNode("School", 180, 180);
    r.addNode("Mall", 300, 180);   r.addNode("Stadium", 440, 180);
    r.addNode("Office", 180, 60);  r.addNode("Bridge", 420, 60);
    r.addNode("Airport", 540, 60);
    String[][] rd = {{"Home","Park St"}, {"Park St","Market"}, {"Market","Beach"},
        {"Home","Library"}, {"Library","School"}, {"Park St","School"}, {"Market","Mall"},
        {"School","Mall"}, {"Mall","Stadium"}, {"Stadium","Beach"}, {"School","Office"},
        {"Office","Bridge"}, {"Bridge","Airport"}, {"Stadium","Airport"}};
    for (String[] e : rd) r.addRoad(e[0], e[1]);
    return r;
}
}

```

A.6 core/NavigationService.java

Route following, simulated GNSS position and turn-by-turn prompts.

```
package ivi.core;

import java.util.List;

/** Navigation service: follows a computed route, simulates GNSS position, issues voice
prompts. */
public class NavigationService {
    private final RouteEngine engine;
    private final EventBus bus;
    private final AudioFocusManager focus;

    private String currentNode = "Home", destination = "";
    private List<String> route = List.of();
    private int seg = 0;
    private double segDist = 0, x, y, promptTimer = 0;
    private boolean navigating = false, promptGiven = false;
    private String instruction = "Not navigating";
    private static final double SPEED = 45; // map units per second

    public NavigationService(RouteEngine engine, EventBus bus, AudioFocusManager focus) {
        this.engine = engine; this.bus = bus; this.focus = focus;
        RouteEngine.Node n = engine.node(currentNode);
        x = n.x; y = n.y;
    }

    public void startRoute(String dest) {
        List<String> r = engine.shortestPath(currentNode, dest);
        if (r.size() < 2) { instruction = "No route to " + dest; return; }
        route = r; destination = dest; seg = 0; segDist = 0; promptGiven = false; navigating
= true;
        instruction = "Head towards " + route.get(1);
        bus.publish(Topics.NAV_ROUTE, String.join(" > ", route));
    }

    public void cancel() { navigating = false; route = List.of(); instruction = "Not navigat-
ing"; }

    public void tick(double dt) {
        if (promptTimer > 0) {
            promptTimer -= dt;
            if (promptTimer <= 0) focus.release(AudioFocusManager.Source.NAV_PROMPT);
        }
        if (!navigating) return;
        RouteEngine.Node a = engine.node(route.get(seg)), b = engine.node(route.get(seg +
1));
        double len = Math.hypot(b.x - a.x, b.y - a.y);
        segDist += SPEED * dt;
        if (!promptGiven && len - segDist <= 45) { promptGiven = true; announce(); }
        if (segDist >= len) {
            currentNode = b.name; seg++; segDist = 0; promptGiven = false;
            x = b.x; y = b.y;
            if (seg >= route.size() - 1) { navigating = false; instruction = "Arrived at " +
destination; }
            return;
        }
        double t = segDist / len;
        x = a.x + (b.x - a.x) * t;
        y = a.y + (b.y - a.y) * t;
    }
}
```

```

    }

    private void announce() {
        String text;
        RouteEngine.Node b = engine.node(route.get(seg + 1));
        if (seg + 2 >= route.size()) {
            text = "Your destination " + b.name + " is ahead";
        } else {
            RouteEngine.Node a = engine.node(route.get(seg)), c = engine.node(route.get(seg +
2));
            double cross = (b.x - a.x) * (c.y - b.y) - (b.y - a.y) * (c.x - b.x); // screen
coords (y down)
            String turn = Math.abs(cross) < 1 ? "Continue straight" : cross > 0 ? "Turn
right" : "Turn left";
            text = turn + " at " + b.name;
        }
        instruction = text;
        bus.publish(Topics.NAV_PROMPT, text);
        focus.request(AudioFocusManager.Source.NAV_PROMPT);
        promptTimer = 3.0;
    }

    public boolean isNavigating() { return navigating; }
    public String instruction() { return instruction; }
    public List<String> route() { return route; }
    public double x() { return x; }
    public double y() { return y; }
    public int speedKmh() { return navigating ? 54 : 0; }
    public String currentNode() { return currentNode; }
    public RouteEngine engine() { return engine; }
}

```

A.7 core/ProjectionManager.java

Smartphone projection session state machine.

```

package ivi.core;

/** Projection manager: smartphone session state machine (Android Auto / Apple CarPlay simu-
lation). */
public class ProjectionManager {
    public enum State { DISCONNECTED, HANDSHAKE, STREAMING }

    private final EventBus bus;
    private final AudioFocusManager focus;
    private State state = State.DISCONNECTED;
    private String protocol = "-", app = "Home";
    private double timer = 0;

    public ProjectionManager(EventBus bus, AudioFocusManager focus) { this.bus = bus; this.-
focus = focus; }

    public void connect(String protocol) {
        if (state != State.DISCONNECTED) return;
        this.protocol = protocol; state = State.HANDSHAKE; timer = 2.0;
        bus.publish(Topics.PROJ_STATE, state + " (" + protocol + ")");
    }

    public void tick(double dt) {
        if (state == State.HANDSHAKE) {
            timer -= dt;
            if (timer <= 0) {

```

```

        state = State.STREAMING; app = "Home";
        bus.publish(Topics.PROJ_STATE, state + " (" + protocol + ")");
    }
}

/** Touch event forwarded to the phone: launches an app on the projected screen. */
public void launchApp(String name) {
    if (state != State.STREAMING) return;
    app = name;
    bus.publish(Topics.PROJ_APP, name);
    if ("Music".equals(name)) focus.request(AudioFocusManager.Source.PROJECTION);
    else focus.release(AudioFocusManager.Source.PROJECTION);
}

public void disconnect() {
    if (state == State.DISCONNECTED) return;
    focus.release(AudioFocusManager.Source.PROJECTION);
    state = State.DISCONNECTED; protocol = "-"; app = "Home";
    bus.publish(Topics.PROJ_STATE, state.toString());
}

public State state() { return state; }
public String protocol() { return protocol; }
public String app() { return app; }
}

```

A.8 ui/MediaPanel.java

Media application screen.

```
package ivi.ui;

import ivi.core.MediaService;
import javax.swing.*.*;
import java.awt.*.*;

/** Media application UI. */
public class MediaPanel extends JPanel {
    private final MediaService media;
    private final JLabel title = new JLabel(), artist = new JLabel(), time = new JLabel(),
    vol = new JLabel();
    private final JProgressBar bar = new JProgressBar(0, 1000);
    private final JButton play = new JButton("Play");
    private final JList<MediaService.Track> list;
    private boolean updating = false;

    public MediaPanel(MediaService media) {
        this.media = media;
        setLayout(new BorderLayout(16, 16));
        setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));

        title.setFont(new Font("SansSerif", Font.BOLD, 28));
        artist.setFont(new Font("SansSerif", Font.PLAIN, 18));
        JPanel info = new JPanel(new GridLayout(0, 1, 4, 4));
        info.add(new JLabel("NOW PLAYING")); info.add(title); info.add(artist);
        info.add(bar); info.add(time);

        JButton prev = new JButton("Prev"), next = new JButton("Next");
        prev.addActionListener(e -> media.prev());
        next.addActionListener(e -> media.next());
        play.addActionListener(e -> media.toggle());
        JSlider slider = new JSlider(0, 100, 60);
        slider.addChangeListener(e -> media.setVolume(slider.getValue()));
        JPanel controls = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
        controls.add(prev); controls.add(play); controls.add(next);
        controls.add(new JLabel(" Volume")); controls.add(slider); controls.add(vol);

        JPanel left = new JPanel(new BorderLayout(0, 20));
        left.add(info, BorderLayout.NORTH); left.add(controls, BorderLayout.CENTER);

        list = new JList<>(media.playlist().toArray(new MediaService.Track[0]));
        list.addListSelectionListener(e -> {
            if (!updating && !e.getValueIsAdjusting() && list.getSelectedIndex() >= 0
                && list.getSelectedIndex() != media.index()) media.select(list.getSe-
lectedIndex());
        });
        JScrollPane sp = new JScrollPane(list);
        sp.setBorder(BorderFactory.createTitledBorder("Playlist"));
        sp.setPreferredSize(new Dimension(280, 100));

        add(left, BorderLayout.CENTER);
        add(sp, BorderLayout.EAST);
    }

    public void refresh() {
        MediaService.Track t = media.current();
        title.setText(t.title); artist.setText(t.artist);
        bar.setValue((int) (1000 * media.position() / t.seconds));
    }
}
```

```

        time.setText(String.format("%d:%02d / %d:%02d   %s", (int) media.position() / 60,
(int) media.position() % 60,
        t.seconds / 60, t.seconds % 60,
        media.isPausedByFocus() ? "[paused by projection audio]" :
media.isPlaying() ? "" : "[paused]"));
        play.setText(media.isPlaying() ? "Pause" : "Play");
        vol.setText("effective: " + media.effectiveVolume());
        updating = true;
        if (list.getSelectedIndex() != media.index()) list.setSelectedIndex(media.index());
        updating = false;
    }
}

```

A.9 ui/NavPanel.java

Navigation application screen with map view.

```

package ivi.ui;

import ivi.core.*;
import javax.swing.*;
import java.awt.*;
import java.util.List;

/** Navigation application UI: destination picker, map view, turn-by-turn instruction. */
public class NavPanel extends JPanel {
    private final NavigationService nav;
    private final JLabel instr = new JLabel(), info = new JLabel();
    private final MapView map = new MapView();

    public NavPanel(NavigationService nav) {
        this.nav = nav;
        setLayout(new BorderLayout(8, 8));
        setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        JComboBox<String> dest = new JComboBox<>();
        for (RouteEngine.Node n : nav.engine().nodes()) if (!n.name.equals("Home")) dest.addItem(n.name);
        dest.setSelectedItem("Airport");
        JButton go = new JButton("Start navigation"), stop = new JButton("Cancel");
        go.addActionListener(e -> nav.startRoute((String) dest.getSelectedItem()));
        stop.addActionListener(e -> nav.cancel());
        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
        top.add(new JLabel("Destination:")); top.add(dest); top.add(go); top.add(stop);
        top.add(info);
        instr.setFont(new Font("SansSerif", Font.BOLD, 20));
        add(top, BorderLayout.NORTH); add(map, BorderLayout.CENTER); add(instr, BorderLayout.SOUTH);
    }

    public void refresh() {
        instr.setText(" " + nav.instruction());
        info.setText("   Speed: " + nav.speedKmh() + " km/h");
        map.repaint();
    }

    private class MapView extends JPanel {
        MapView() { setBackground(new Color(24, 30, 38)); }
        @Override protected void paintComponent(Graphics g0) {
            super.paintComponent(g0);
            Graphics2D g = (Graphics2D) g0;
            g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

```

```

        double sx = Math.min((getWidth() - 60) / 560.0, (getHeight() - 60) / 300.0);
        RouteEngine e = nav.engine();
        g.setStroke(new BasicStroke(5f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
        g.setColor(new Color(70, 80, 95));
        for (String[] r : e.roads()) line(g, e.node(r[0]), e.node(r[1]), sx);
        List<String> route = nav.route();
        g.setColor(new Color(60, 160, 255));
        for (int i = 0; i + 1 < route.size(); i++) line(g, e.node(route.get(i)),
e.node(route.get(i + 1)), sx);
        g.setFont(new Font("SansSerif", Font.PLAIN, 12));
        for (RouteEngine.Node n : e.nodes()) {
            int px = px(n.x, sx), py = py(n.y, sx);
            g.setColor(new Color(200, 205, 215)); g.fillOval(px - 5, py - 5, 10, 10);
            g.drawString(n.name, px + 8, py - 8);
        }
        int vx = px(nav.x(), sx), vy = py(nav.y(), sx);
        g.setColor(new Color(255, 190, 40)); g.fillOval(vx - 8, vy - 8, 16, 16);
        g.setColor(Color.WHITE); g.drawOval(vx - 8, vy - 8, 16, 16);
    }
    private int px(double x, double s) { return (int) (30 + x * s); }
    private int py(double y, double s) { return (int) (30 + y * s); }
    private void line(Graphics2D g, RouteEngine.Node a, RouteEngine.Node b, double s) {
        g.drawLine(px(a.x, s), py(a.y, s), px(b.x, s), py(b.y, s));
    }
}
}

```

A.10 ui/ProjectionPanel.java

Projection screen with simulated phone display.

```

package ivi.ui;

import ivi.core.ProjectionManager;
import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

/** Projection UI: simulated smartphone session (Android Auto / Apple CarPlay). */
public class ProjectionPanel extends JPanel {
    private static final String[] APPS = {"Maps", "Music", "Phone", "Messages"};
    private final ProjectionManager proj;
    private final JLabel state = new JLabel();
    private final Screen screen = new Screen();

    public ProjectionPanel(ProjectionManager proj) {
        this.proj = proj;
        setLayout(new BorderLayout(8, 8));
        setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        JButton aa = new JButton("Connect Android Auto"), cp = new JButton("Connect Apple
CarPlay"),
            off = new JButton("Disconnect");
        aa.addActionListener(e -> proj.connect("Android Auto"));
        cp.addActionListener(e -> proj.connect("Apple CarPlay"));
        off.addActionListener(e -> proj.disconnect());
        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
        top.add(aa); top.add(cp); top.add(off); top.add(state);
        add(top, BorderLayout.NORTH); add(screen, BorderLayout.CENTER);
    }

    public void refresh() {

```

```

        state.setText("    State: " + proj.state() + "    Protocol: " + proj.protocol());
        screen.repaint();
    }

    private class Screen extends JPanel {
        Screen() {
            setBackground(new Color(24, 30, 38));
            addMouseListener(new MouseAdapter() {
                @Override public void mousePressed(MouseEvent e) {
                    for (int i = 0; i < APPS.length; i++) if (tile(i).contains(e.getPoint()))
proj.launchApp(APPS[i]);
                }
            });
        }
        private Rectangle tile(int i) {
            int w = 150, h = 90, gap = 20, x0 = (getWidth() - (w * 4 + gap * 3)) / 2;
            return new Rectangle(x0 + i * (w + gap), 60, w, h);
        }
        @Override protected void paintComponent(Graphics g0) {
            super.paintComponent(g0);
            Graphics2D g = (Graphics2D) g0;
            g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_AN-
TIALIAS_ON);
            g.setColor(Color.WHITE);
            if (proj.state() != ProjectionManager.State.STREAMING) {
                g.setFont(new Font("SansSerif", Font.PLAIN, 22));
                String m = proj.state() == ProjectionManager.State.HANDSHAKE
                    ? "Connecting to phone (" + proj.protocol() + ") ..." : "No phone
connected. Choose a protocol above.";
                g.drawString(m, 40, 60);
                return;
            }
            g.setFont(new Font("SansSerif", Font.BOLD, 18));
            g.drawString(proj.protocol() + " - projected phone screen (touch a tile)", 40,
35);

            for (int i = 0; i < APPS.length; i++) {
                Rectangle r = tile(i);
                boolean sel = APPS[i].equals(proj.app());
                g.setColor(sel ? new Color(60, 160, 255) : new Color(60, 70, 85));
                g.fillRoundRect(r.x, r.y, r.width, r.height, 16, 16);
                g.setColor(Color.WHITE);
                g.drawString(APPS[i], r.x + 20, r.y + 52);
            }
            g.setFont(new Font("SansSerif", Font.PLAIN, 22));
            String a = proj.app();
            String body = a.equals("Music") ? "Phone music playing (IVI media paused)"
                : a.equals("Home") ? "Select an app" : a + " running on phone";
            g.drawString(body, 40, 210);
        }
    }
}

```

A.11 ui/IviSystem.java

Wires services and panels together; owns the simulation step.

```

package ivi.ui;

import ivi.core.*;
import javax.swing.*;
import java.awt.*;

```

```

/** Wires the services and UI panels together (the "head unit"). */
public class IviSystem {
    public final EventBus bus = new EventBus();
    public final AudioFocusManager focus = new AudioFocusManager(bus);
    public final MediaService media = new MediaService(bus, focus);
    public final NavigationService nav = new NavigationService(RouteEngine.sample(), bus, fo-
cus);
    public final ProjectionManager proj = new ProjectionManager(bus, focus);
    public final JPanel root = new JPanel(new BorderLayout());
    public final JTabbedPane tabs = new JTabbedPane();

    private final MediaPanel mediaPanel = new MediaPanel(media);
    private final NavPanel navPanel = new NavPanel(nav);
    private final ProjectionPanel projPanel = new ProjectionPanel(proj);
    private final JLabel status = new JLabel();
    private final JTextArea log = new JTextArea(6, 40);

    public IviSystem() {
        tabs.addTab("Media", mediaPanel);
        tabs.addTab("Navigation", navPanel);
        tabs.addTab("Projection", projPanel);
        log.setEditable(false);
        log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
        JScrollPane sp = new JScrollPane(log);
        sp.setBorder(BorderFactory.createTitledBorder("Event bus log (data flows)"));
        status.setBorder(BorderFactory.createEmptyBorder(4, 10, 4, 10));
        JPanel south = new JPanel(new BorderLayout());
        south.add(status, BorderLayout.NORTH); south.add(sp, BorderLayout.CENTER);
        root.add(tabs, BorderLayout.CENTER); root.add(south, BorderLayout.SOUTH);
        bus.addListener((t, p) -> {
            log.append(String.format("%-18s %s\n", t, p));
            if (log.getLineCount() > 200) log.setText("");
            log.setCaretPosition(log.getDocument().getLength());
        });
        refresh();
    }

    /** Advance simulation by dt seconds and refresh the screens. */
    public void step(double dt) {
        media.tick(dt); nav.tick(dt); proj.tick(dt);
        refresh();
    }

    private void refresh() {
        mediaPanel.refresh(); navPanel.refresh(); projPanel.refresh();
        status.setText("Audio owner: " + focus.owner() + " | Media focus: " + focus.medi-
aFocus());
    }
}

```

A.12 ui/MainApp.java

Application entry point and clock.

```

package ivi.ui;

import javax.swing.*;

/** Entry point: opens the IVI head-unit simulator window. */
public class MainApp {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {

```

```
IviSystem ivi = new IviSystem();
JFrame f = new JFrame("IVI Simulator - Media | Navigation | Projection");
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
f.setContentPane(ivi.root);
f.setSize(980, 680);
f.setLocationRelativeTo(null);
f.setVisible(true);
new Timer(250, e -> ivi.step(0.25)).start();
});
}
```

A.13 test/IviTests.java

Assertion-based tests of the core logic.

```
package ivi.test;

import ivi.core.*;
import java.util.List;

/** Simple assertion-based tests (run: java -cp out ivi.test.IviTests). */
public class IviTests {
    static int passed = 0;
    static void check(boolean c, String name) {
        if (!c) throw new AssertionError("FAILED: " + name);
        passed++; System.out.println("PASS " + name);
    }

    public static void main(String[] a) {
        RouteEngine r = RouteEngine.sample();
        check(r.shortestPath("Home", "Airport").equals(
            List.of("Home", "Park St", "School", "Mall", "Stadium", "Airport")), "Dijkstra Home->Airport");

        EventBus bus = new EventBus();
        AudioFocusManager af = new AudioFocusManager(bus);
        MediaService m = new MediaService(bus, af);
        NavigationService nav = new NavigationService(r, bus, af);
        ProjectionManager pm = new ProjectionManager(bus, af);

        m.play();
        check(m.isPlaying() && m.effectiveVolume() == 60, "media plays at full volume");
        af.request(AudioFocusManager.Source.NAV_PROMPT);
        check(m.effectiveVolume() == 18, "nav prompt ducks media");
        af.release(AudioFocusManager.Source.NAV_PROMPT);
        check(m.effectiveVolume() == 60, "media volume restored");

        pm.connect("Android Auto");
        check(pm.state() == ProjectionManager.State.HANDSHAKE, "projection handshake");
        for (int i = 0; i < 10; i++) pm.tick(0.25);
        check(pm.state() == ProjectionManager.State.STREAMING, "projection streaming");
        pm.launchApp("Music");
        check(!m.isPlaying() && m.isPausedByFocus(), "phone music pauses IVI media");
        pm.disconnect();
        check(m.isPlaying(), "media resumes after disconnect");

        int[] prompts = {0};
        bus.subscribe(Topics.NAV_PROMPT, p -> prompts[0]++);
        nav.startRoute("Airport");
        for (int i = 0; i < 2000 && nav.isNavigating(); i++) nav.tick(0.25);
        check(!nav.isNavigating() && nav.instruction().startsWith("Arrived"), "navigation
reaches destination");
        check(prompts[0] >= 4, "turn-by-turn prompts issued (" + prompts[0] + ")");
        for (int i = 0; i < 20; i++) nav.tick(0.25);
        check(af.mediaFocus() == AudioFocusManager.Focus.GAIN, "focus released after
prompts");
        System.out.println(passed + " tests passed");
    }
}
```

A.14 test/ScreenshotTool.java

Headless renderer that produces the report screenshots.

```

package ivi.test;

import ivi.ui.IviSystem;
import javax.imageio.ImageIO;
import javax.swing.*;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;

/** Renders the UI to PNG files for the report (works without a display:
java.awt.headless=true). */
public class ScreenshotTool {
    public static void main(String[] args) throws Exception {
        System.setProperty("java.awt.headless", "true");
        File dir = new File(args.length > 0 ? args[0] : "screenshots");
        dir.mkdirs();
        SwingUtilities.invokeLater(() -> {
            try {
                IviSystem s = new IviSystem();
                s.root.setSize(980, 680);
                s.media.play(); s.step(3);
                shot(s, 0, "01_media_playing", dir);

                s.tabs.setSelectedIndex(1);
                s.nav.startRoute("Airport");
                for (int i = 0; i < 4; i++) s.step(0.25);
                shot(s, 1, "02_navigation_route", dir);

                for (int i = 0; i < 400 && s.focus.mediaFocus() != ivi.core.AudioFocusManager.Focus.DUCK; i++) s.step(0.25);
                shot(s, 1, "03_navigation_prompt_media_ducked", dir);

                s.nav.cancel(); s.step(4);
                s.tabs.setSelectedIndex(2);
                s.proj.connect("Android Auto"); s.step(0.5);
                shot(s, 2, "04_projection_handshake", dir);
                s.step(2); s.proj.launchApp("Maps");
                shot(s, 2, "05_projection_streaming", dir);
                s.proj.launchApp("Music"); s.step(0.5);
                shot(s, 2, "06_projection_audio_media_paused", dir);
                System.out.println("Screenshots saved in " + dir.getAbsolutePath());
            } catch (Exception e) { throw new RuntimeException(e); }
        });
    }

    static void shot(IviSystem s, int tab, String name, File dir) throws Exception {
        s.tabs.setSelectedIndex(tab);
        layout(s.root);
        BufferedImage img = new BufferedImage(980, 680, BufferedImage.TYPE_INT_RGB);
        Graphics2D g = img.createGraphics();
        s.root.paint(g); g.dispose();
        ImageIO.write(img, "png", new File(dir, name + ".png"));
    }

    static void layout(Component c) {
        if (c instanceof Container) {
            ((Container) c).doLayout();
            for (Component ch : ((Container) c).getComponents()) layout(ch);
        }
    }
}

```

Appendix B: Detailed reference architecture

This diagram shows the complete reference architecture, including the vehicle network, external services, cloud and security boundary. The simulator implements the media, navigation and projection parts. Interfaces and protocols in the diagram are representative.

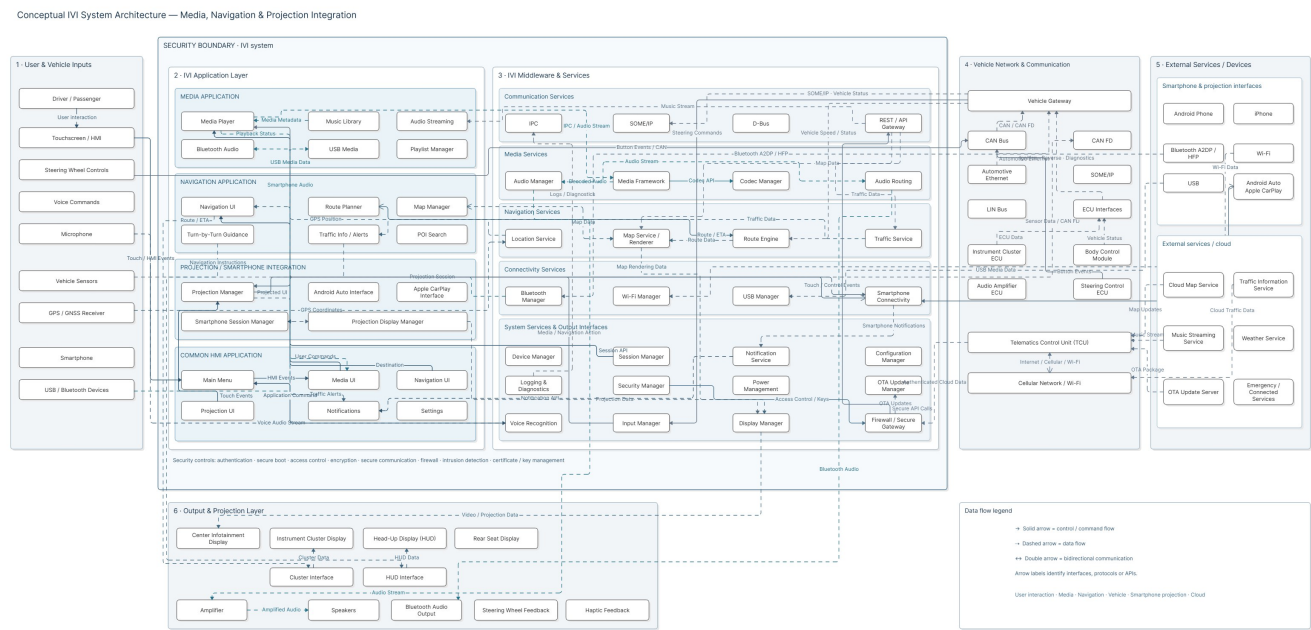


Figure B1: Detailed reference architecture of the IVI system (zoom the digital copy to read labels)